

Assignment – 8

Implement and analyze various methods of studying correlation and regression including Karl Pearson's coefficient and simple linear regression.

✓ THEORY

Understanding the relationship between variables is the bread and butter of data science. Here is a breakdown of your questions, keeping things clear and statistically sound.

1. What is correlation? How is it different from regression?

Correlation measures the strength and direction of a linear relationship between two variables. It is expressed by a coefficient (usually Pearson's r) ranging from -1 to +1.

Regression goes a step further by describing the *functional* relationship between variables. It allows you to predict the value of a dependent variable (Y) based on the value of an independent variable (X).

Feature	Correlation	Regression
Purpose	To find a numerical value expressing the relationship.	To explain the impact of cha
Symmetry	Symmetric: Correlation between X and Y is the same as Y and X .	Asymmetric: Predicting Y fr
Output	A single point (coefficient).	An equation (a line or curve)
Cause/Effect	Does not imply causation.	Often used to model a math

2. What is simple linear regression? Explain with suitable example.

Simple Linear Regression (SLR) is a statistical method that allows us to summarize and study relationships between two continuous variables. It models the relationship using a straight line.

The formula is:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

- Y : The Dependent Variable (what you want to predict).
- X : The Independent Variable (the predictor).
- β_0 : The Y-intercept.
- β_1 : The Slope (the change in Y for every 1 unit change in X).
- ϵ : The error term (residuals).

Example: Imagine you are a coffee shop owner. You want to predict **daily revenue (Y)** based on the **outside temperature (X)**. If your regression model is $Y = 50 + 20X$, it means that even if the temperature is 0, you expect to make 50. *For every degree the temperature rises, your revenue increases by 20.*

To calculate the Pearson correlation in Python using the approach from the provided code, you should first ensure your data is cleaned (handling missing values) and then use the `scipy.stats` library.

Here is the modified answer tailored to your camera dataset:

3. How do you calculate Pearson correlation in Python?

In the Python ecosystem, you typically use **pandas** for quick checks or **scipy** for more detailed statistical analysis.

Using Pandas: This is the fastest way to get a single correlation value or a full matrix from your dataframe.

```
# Returns a single correlation value between resolution and price
correlation = df['Max resolution'].corr(df['Price'])
print(f"Pearson Correlation: {correlation:.4f}")
```

Using Scipy (to get the p-value): As seen in the provided code, the `pearsonr` function from `scipy.stats` is preferred when you need to check the statistical significance (p-value). It is crucial to drop `NaN` (missing) values first to avoid errors.

```
from scipy.stats import pearsonr

# 1. Clean the data (remove rows where either value is missing)
df_clean = df[['Max resolution', 'Price']].dropna()

# 2. Calculate the coefficient (r) and the p-value
corr_coeff, p_value = pearsonr(df_clean['Max resolution'], df_clean['Price'])

print(f"Karl Pearson Correlation Coefficient: {corr_coeff:.4f}")
print(f"P-value: {p_value:.4f}")
```

Understanding the Result

- **Correlation Coefficient (r):** In our data, the result was **0.1832**. This indicates a **weak positive relationship**, meaning as resolution increases, price tends to increase only slightly.
- **P-value:** This tells you if the correlation is statistically significant. If it is less than 0.05, you can be confident the relationship is not due to random chance.

4. What is multicollinearity? Does it affect simple regression?

Multicollinearity occurs when two or more independent variables in a multiple regression model are highly correlated with each other. This is a headache because it makes it difficult for the model to determine which variable is actually doing the "heavy lifting" for the prediction.

Does it affect Simple Regression? No. By definition, Simple Linear Regression involves only **one** independent variable. Since there isn't a second independent variable to be correlated with, multicollinearity cannot exist in a simple regression setup.

5. A dataset shows strong correlation but poor prediction accuracy. Why?

It sounds counterintuitive, but a high correlation coefficient doesn't always guarantee a perfect model. Here is why:

- **Non-linearity:** Pearson correlation measures *linear* relationships. If your data follows a complex curve, the "strong" linear correlation might not capture the full picture, leading to a poor linear fit.
- **Outliers:** A few extreme data points can pull the correlation coefficient higher (or lower) than it should be, while the rest of the data fails to follow the trend line closely.
- **High Variance (Noise):** You might have a clear upward trend (high correlation), but if the data points are widely scattered around that trend line, the individual predictions will have high error.
- **The "Hidden Factor":** You might have a high correlation between X and Y , but Y might actually be influenced by five other variables you aren't measuring. Without those, your model is "underfitted."

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import pearsonr
from sklearn.linear_model import LinearRegression

from google.colab import drive
drive.mount('/content/drive')

file_path = '/content/drive/My Drive/camera_dataset.csv'
```

```
df = pd.read_csv(file_path)

df_clean = df[['Max resolution', 'Price']].dropna()

corr_coeff, p_value = pearsonr(df_clean['Max resolution'], df_clean['Price'])
print(f"Karl Pearson Correlation Coefficient: {corr_coeff:.4f}")

X = df_clean[['Max resolution']]
y = df_clean['Price']

model = LinearRegression()
model.fit(X, y)

slope = model.coef_[0]
intercept = model.intercept_
r_squared = model.score(X, y)

print(f"Regression Equation: Price = ({slope:.4f} * Max resolution) + {intercept:.4f}")
print(f"R-squared Value: {r_squared:.4f}")

plt.figure(figsize=(10, 6))
sns.scatterplot(x='Max resolution', y='Price', data=df_clean, label='Data Points')
plt.plot(df_clean['Max resolution'], model.predict(X), color='red', linewidth=2)
plt.title('Correlation & Regression: Max Resolution vs Price')
plt.xlabel('Max Resolution')
plt.ylabel('Price ($)')
plt.legend()
plt.show()
```

Mounted at /content/drive

Karl Pearson Correlation Coefficient: 0.1832

Regression Equation: Price = (0.1834 * Max resolution) + 3.4576

R-squared Value: 0.0336

Correlation & Regression: Max Resolution vs Price

